ARTIFICIAL INTELLIGENCE | RETRIEVAL-AUGMENTED GENERATION

Industrial Product Intelligence Assistant

Project Overview, System Architecture and Technical Design

Conversational search over industrial catalogues, datasheets and manuals, grounded in local document evidence and clearly separated from online sources.

FastAPI | PyMuPDF | Ollama | ChromaDB | OpenRouter

DOCUMENT TYPE	VERSION	STATUS	APPROACH
Technical Project Report	1.0	Prototype Development	RAG

Project category: Artificial Intelligence | Retrieval-Augmented Generation (RAG) | Industrial Product Intelligence

Contents

01 Executive Summary . . . 3
02 Problem Statement . . . 3
03 Project Objectives . . . 4
04 Proposed Solution . . . 4
05 Technology Stack . . . 5
06 System Architecture . . . 6
07 Document Ingestion Workflow . . . 7
08 Question Answering Workflow . . . 8
09 Core Functional Modules . . . 9
10 Example Use Case: Understanding an FLP Motor . . . 10
11 Data Storage and Information Flow . . . 10
12 Security and Reliability Considerations . . . 11
13 Current Prototype Scope . . . 12
14 Known Limitations and Challenges . . . 12
15 Future Enhancements . . . 12
16 Testing and Validation Plan . . . 13
17 Success Criteria . . . 14
18 Conclusion . . . 14

About this document

This report describes the purpose, architecture, workflows, security considerations and validation plan of the Industrial Product Intelligence Assistant. It reflects the **intended** design of a prototype; each capability must be verified against the running implementation before the project is presented as fully operational.

01 Executive Summary

The Industrial Product Intelligence Assistant is an AI-powered application that helps users understand industrial products, retrieve specifications from technical documents, and obtain clear answers to product-related questions.

Industrial product information is spread across catalogues, datasheets, installation manuals, operation guides and manufacturer documentation. Searching these documents manually is slow, especially for users who need a specific technical detail but are unfamiliar with industrial terminology. This project addresses the problem with a conversational interface that combines document-based retrieval with large language model capabilities.

Users upload PDF documents and ask questions in natural language. The system extracts text, generates embeddings with a locally running Ollama model, and stores the vectors in ChromaDB. When a question is submitted, the application retrieves the relevant passages and passes them to a language model hosted through OpenRouter, which produces a readable answer. An online web-search capability is planned for cases where the uploaded documents are insufficient.

The goal is to make industrial product information more accessible, searchable and understandable, while keeping **internal document evidence** clearly separate from **information obtained online**.

RAG	Local	Hosted	PDF
Architecture pattern	Embeddings via Ollama	LLM via OpenRouter	Knowledge base format

02 Problem Statement

Industrial product information is frequently stored in large, unstructured PDF documents, making it difficult to locate specific details efficiently. A user researching an industrial motor, for example, may need its operating principle, applications, power rating, protection class, installation requirements or suitability for a particular environment. Finding these details may require reviewing many pages across several documents.

Traditional keyword search struggles with natural-language questions, synonyms, and questions that combine information from different passages. Relying exclusively on a general-purpose AI model is also risky, because answers may not be supported by the manufacturer's documentation.

Problem statement

To develop an AI-powered industrial product assistant that enables users to ask natural-language questions, retrieve relevant information from uploaded technical documents, and receive clear, evidence-oriented answers, with online search available as a supplementary information source.

03 Project Objectives

#	Objective	Description
1	Simplify technical information access	Let users ask questions in everyday language instead of searching documents manually.
2	Support PDF knowledge bases	Accept industrial product catalogues, manuals and datasheets in PDF format.
3	Implement semantic search	Retrieve passages by meaning rather than relying only on exact keyword matches.
4	Generate contextual answers	Use retrieved document evidence to support answers generated by an LLM.
5	Enable online research	Support web search when internal documentation is missing or insufficient.
6	Maintain data control	Run document embeddings locally through Ollama and keep the vector database on the local machine.
7	Improve transparency	Distinguish document-based evidence from online information and state when information cannot be verified.
8	Provide an accessible interface	Deliver a browser-based chat experience through a lightweight HTML, CSS and JavaScript frontend.

04 Proposed Solution

The application follows a **Retrieval-Augmented Generation (RAG)** architecture, which combines two capabilities: **Retrieval**, which finds relevant passages from the user's indexed documents, and **Generation**, which uses a language model to formulate an answer from those passages and the question. The solution runs through five main stages.

Stage	Description
1 Document ingestion	The user uploads a PDF. The backend extracts its text and divides it into manageable chunks.
2 Embedding generation	Chunks are converted into numerical vectors using the local `nomic-embed-text` model through Ollama.
3 Vector storage	Vectors, text chunks and document metadata are stored in ChromaDB.
4 Question answering	The question is embedded with the same model. ChromaDB retrieves semantically relevant passages, which are supplied to the LLM through OpenRouter.
5 Response generation	The application returns a natural-language answer. When enabled and supported by the model and API, web search can supplement insufficient internal evidence.

05 Technology Stack

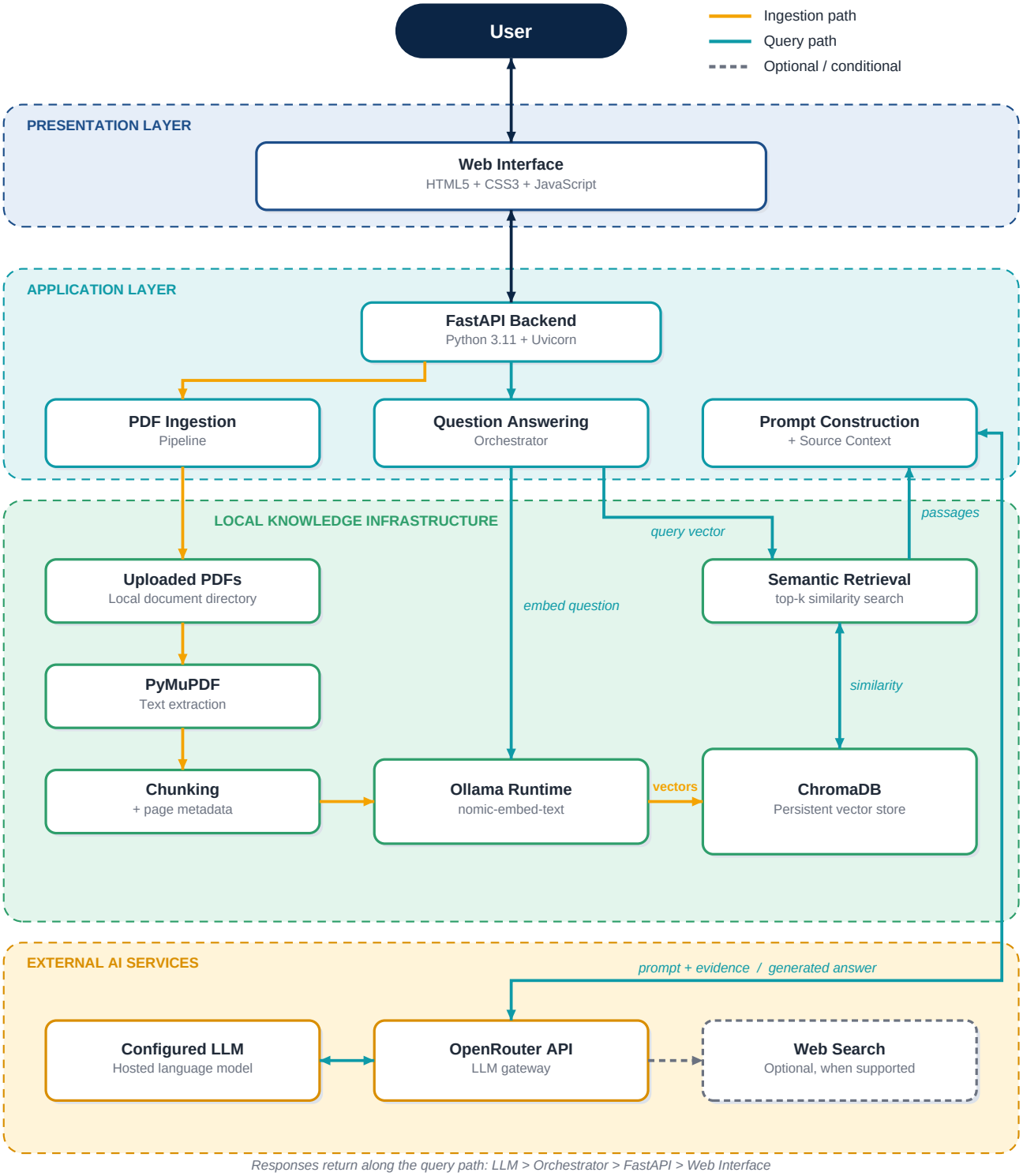
Layer	Technology	Responsibility
Frontend	HTML5, CSS3, JavaScript	Chat interface, PDF upload and response display
Backend API	Python 3.11, FastAPI	Request handling, document ingestion and chat orchestration
ASGI server	Uvicorn	Runs the FastAPI application
PDF processing	PyMuPDF	Extracts text from PDF pages
Embedding model	`nomic-embed-text`	Converts text into semantic vectors
Local model runtime	Ollama	Runs the embedding model locally
Vector database	ChromaDB	Stores and searches document embeddings
LLM access	OpenRouter API	Connects the backend to a hosted language model
Online research	OpenRouter web-search tool (when supported)	Supplements internal evidence with web information
Configuration	Environment variables and `.env`	Stores application configuration and API credentials
Persistence	Local filesystem	Stores uploaded PDFs and the ChromaDB database

Why these technologies?

Technology	Rationale
FastAPI	Lightweight Python API with request validation and a convenient development workflow.
Ollama	Enables local embedding generation without sending document chunks to an external embedding service.
ChromaDB	Persistent vector storage and similarity search without a separately managed database server.
OpenRouter	Access to hosted language models through a common API interface.
HTML / CSS / JavaScript	Lightweight, customizable browser interface without a separate frontend framework.
PyMuPDF	Extracts text from PDF pages for subsequent chunking and indexing.

06 System Architecture

The diagram shows the four layers of the system and the two main workflows: document ingestion (amber) and question answering (teal).



Responses return along the query path: LLM > Orchestrator > FastAPI > Web Interface

Figure 1. Layered architecture with ingestion and query paths

Architecture explanation

Layer	Role
Presentation layer	Provides the interface for uploading PDFs, entering questions and viewing responses.
Application layer	FastAPI coordinates file uploads, ingestion, retrieval, LLM requests and response delivery.
Local knowledge infrastructure	PDF text is extracted, split into chunks, embedded locally and stored in ChromaDB. At query time, relevant chunks are retrieved through vector similarity search.
External AI services	OpenRouter supplies the configured language model. The optional web-search capability can provide additional evidence where supported by the API request and selected model.

Important design principle

Document **embedding** is local, but the question-answering request is sent to the external LLM service. Retrieved document excerpts may therefore be transmitted to that service as part of the prompt. **The design is not entirely offline.**

07 Document Ingestion Workflow

The ingestion workflow prepares uploaded technical documents for semantic search.

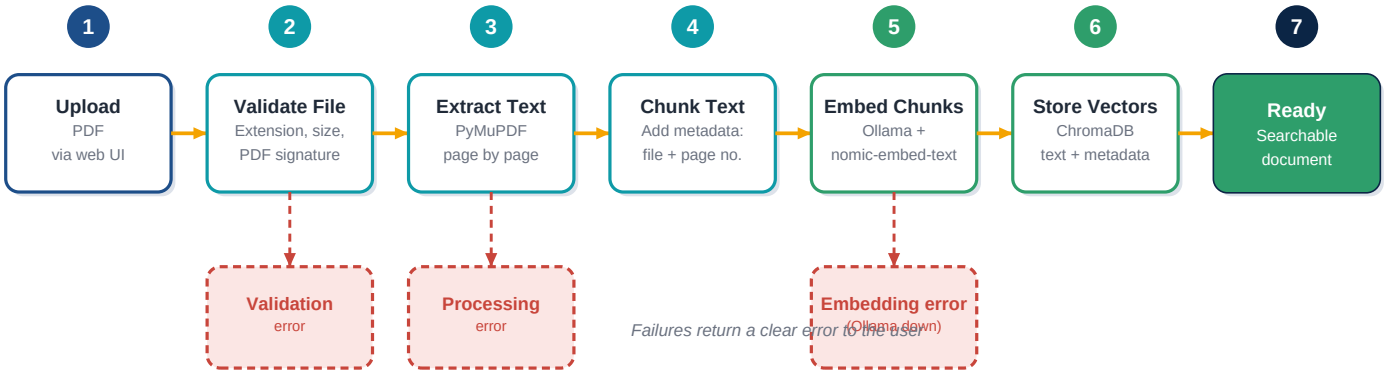


Figure 2. Ingestion pipeline with failure branches

1. The user selects and uploads a PDF through the web interface.
2. FastAPI validates the file extension, size and basic PDF signature.
3. The PDF is saved to the configured local document directory.
4. PyMuPDF extracts the text from each page.
5. The text is divided into smaller chunks suitable for embedding and retrieval.
6. Each chunk is associated with metadata such as source filename and page number.
7. Ollama generates embeddings using `nomic-embed-text`.
8. ChromaDB stores the vectors, text and metadata.
9. The indexed content becomes available for subsequent questions.

08 Question Answering Workflow

When a question is submitted, the application retrieves relevant document passages before generating the answer.

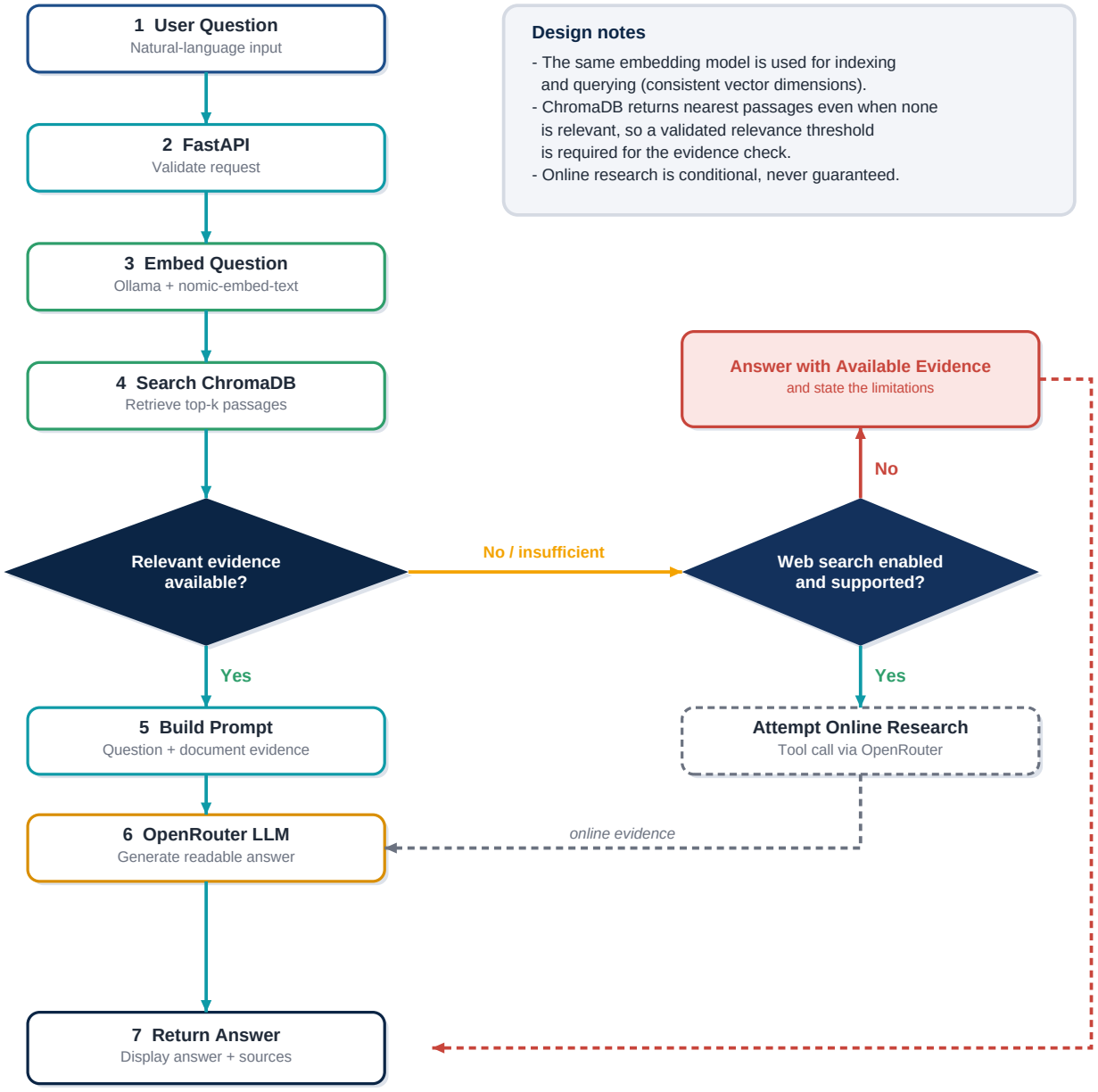


Figure 3. Query flow with evidence check and web-search fallback

1. The user submits a natural-language question.
2. FastAPI validates the request.
3. Ollama generates an embedding for the question.
4. ChromaDB searches for similar document vectors.
5. The application collects the retrieved text and source metadata.
6. The backend constructs a prompt with the question, relevant evidence and answer-generation instructions.
7. OpenRouter sends the request to the configured language model.
8. If web search is enabled and supported, online research may supplement the response.
9. The backend returns the answer and available source information to the frontend.

Implementation note
The diagram represents the intended workflow. Evidence-sufficiency assessment, source attribution and graceful fallback must be implemented and tested for the application to behave this way.

09 Core Functional Modules

9.1 PDF Upload and Ingestion

- Accept PDF uploads and validate files before processing.
- Extract text and preserve page-level metadata.
- Generate and persist embeddings.
- Return ingestion status and indexed-chunk information.

9.2 Local Embedding Service

- Connect to the Ollama service and load the configured embedding model.
- Generate vectors for document chunks and user questions.
- Use the **same model** for indexing and querying, which is essential for consistent vector dimensions and compatible semantic retrieval.

9.3 Vector Retrieval

- Query ChromaDB using the question embedding and retrieve the configured number of passages.
- Return text, metadata and similarity-distance information; handle an empty knowledge base and retrieval errors.
- A vector database can return the nearest passages even when none is relevant, so a production implementation needs a validated relevance threshold or other evidence-quality check.

9.4 LLM Answer Generation

- Submit the question and retrieved evidence to OpenRouter.
- Follow instructions against inventing specifications or certifications.
- Explain technical terms in accessible language and distinguish verified evidence from assumptions.
- State when the available evidence is insufficient.

9.5 Online Search Fallback

- Trigger web research when appropriate and prefer manufacturer websites and official technical documentation.
- Preserve source URLs or citations returned by the search tool and distinguish online information from uploaded-document evidence.
- Avoid presenting unsupported search results as verified specifications.
- Treat web search as conditional, not guaranteed. If the tool fails, disclose the limitation rather than imply online verification occurred.

9.6 Health and Status Monitoring

- The health endpoint reports application status, model configuration, API-key configuration status and indexed-chunk count.
- A production check should also verify the backend, Ollama connectivity, embedding-model availability, ChromaDB accessibility, OpenRouter connectivity and web-search capability.
- A health response alone is not proof that all dependent services are operational.

10 Example Use Case: Understanding an FLP Motor

An FLP motor is commonly understood in industrial contexts as a **flameproof motor** designed for specified hazardous environments. Its suitability depends on the applicable protection marking, certification, installation conditions and hazardous-area classification.

A user might ask:

> *"Explain FLP motors in simple language, including their uses."*

Intended assistant workflow

1. Search uploaded motor catalogues and datasheets for FLP-related information.
2. Retrieve relevant descriptions, ratings, certifications and application details.
3. Generate a simple explanation based on the retrieved evidence.
4. If the documents lack enough information, use supported online research when enabled.
5. Clearly identify the source of each important claim.

Potential answer topics

- What an FLP motor is
- The meaning of flameproof protection
- Typical industrial applications
- Available power ratings and enclosure specifications
- Applicable hazardous-area certifications
- Installation and maintenance considerations

Safety-critical caution

The application must not assume that every FLP motor is suitable for every hazardous location. Users should verify the exact motor marking and manufacturer documentation before making safety-critical decisions.

11 Data Storage and Information Flow

The prototype uses local persistence for documents and vectors. The diagram highlights which data stays on the local machine and which leaves it when an answer is generated.

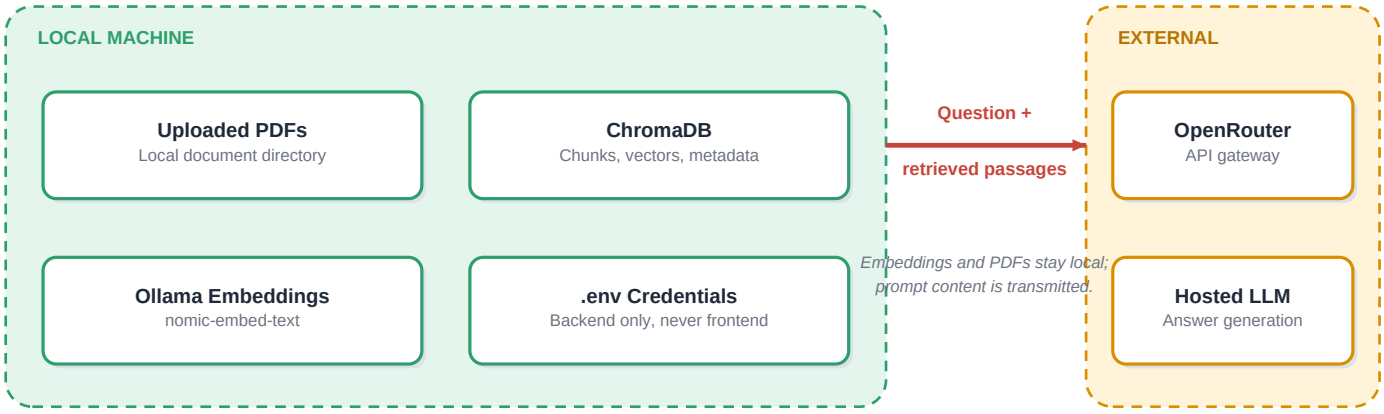


Figure 4. Local data boundary and external transmission

Data	Storage or destination	Purpose
Uploaded PDFs	Local document directory	Original source material
Extracted text chunks	ChromaDB	Searchable document content
Chunk embeddings	ChromaDB	Semantic similarity search
Source metadata	ChromaDB	Document and page identification
API credentials	Local `.env` configuration	Authenticated access to OpenRouter
User question and retrieved context	OpenRouter request when generating an answer	LLM response generation

The exact information retained by external services depends on their service policies and configuration. API credentials must never appear in frontend JavaScript, source-control repositories, logs or generated reports.

12 Security and Reliability Considerations

API credential management

- Keep the OpenRouter key in the backend environment.
- Exclude `.env` from version control.
- Replace any key that has been exposed.
- Avoid logging secrets or authorization headers.

File upload security

- Enforce file-size limits and PDF validation.
- Generate safe server-side filenames.
- Consider duplicate handling and malicious-file scanning.
- Do not assume a .pdf extension guarantees a safe document.

Document processing

- Handle scanned PDFs that lack extractable text.
- Report empty or failed extraction clearly.
- Consider OCR as a future enhancement.
- Track source pages to support evidence verification.

AI response reliability

- Avoid inventing specifications, prices or certifications.
- Include source information for factual claims.
- Identify missing or conflicting evidence.
- Treat retrieved text as untrusted data, not instructions.

Service resilience

- Detect unavailable Ollama or OpenRouter services.
- Use timeouts and clear error messages.
- Handle an unavailable vector database gracefully.
- Allow online-only answers if enabled and supported.

Privacy

- State that files and embeddings are stored locally.
- State that questions and retrieved passages may be sent to OpenRouter.
- Do not call the app fully offline or private without validating the complete data flow.

13 Current Prototype Scope

- Browser-based conversational interface.
- PDF upload and local document storage.
- PDF text extraction and chunking.
- Local embedding generation through Ollama.
- Persistent vector storage in ChromaDB.
- Semantic retrieval of document passages.
- Answer generation through OpenRouter.
- Optional web-search integration.
- API health and indexing-status reporting.

Verification required
These capabilities describe the intended application scope. Each must be verified against the running implementation before the project is presented as fully operational.

14 Known Limitations and Challenges

Area	Limitation
Local model availability	Indexing and semantic retrieval depend on Ollama and the configured embedding model. If Ollama is unavailable, the retrieval path may fail before the application reaches OpenRouter.
Retrieval quality	Semantic similarity does not guarantee factual relevance. Chunk size, overlap, metadata, embedding quality and relevance thresholds influence results.
PDF quality	Scanned pages, tables, diagrams and complex layouts may not be represented adequately by plain-text extraction alone.
External LLM dependency	Answer generation depends on internet access, OpenRouter availability, API credits and the selected model.
Online search reliability	Enabling web search in the interface does not guarantee a search is executed. API configuration, tool compatibility and citation extraction must be tested.
Source verification	Source attribution and the distinction between retrieved evidence and model-generated explanation require deliberate implementation and testing.
Safety-critical applications	AI responses support research and document navigation. They do not replace certified engineering assessments, hazardous-area classification or official manufacturer instructions.

15 Future Enhancements

#	Enhancement	Description
1	Hybrid retrieval	Combine vector similarity with keyword retrieval to improve exact model-number and specification searches.
2	Evidence-quality assessment	Apply relevance thresholds and explicit checks for insufficient or contradictory evidence.
3	Source citations	Display document names, page numbers, quoted passages and clickable online sources.

#	Enhancement	Description
4	OCR support	Extract information from scanned manuals and image-based catalogues.
5	Table extraction	Improve retrieval of electrical ratings, dimensions, performance curves and specification tables.
6	Multiple knowledge bases	Organize documents by manufacturer, product category or customer.
7	Conversation history	Allow follow-up questions while retaining the appropriate source context.
8	Comparison mode	Compare products across ratings, certifications, dimensions and applications.
9	Evaluation framework	Measure retrieval relevance, factual accuracy, citation correctness and latency with a curated test dataset.
10	Deployment hardening	Add authentication, access control, rate limiting, structured logs, monitoring and secure configuration.
11	Operational fallback	Allow online-only responses when the local embedding service is unavailable, with clear disclosure that uploaded documents were not searched.
12	Cost and performance monitoring	Track LLM usage, response times, embedding throughput and failures.

16 Testing and Validation Plan

Test	Expected outcome
Upload a valid, text-based PDF	Document is indexed successfully
Upload a non-PDF file	Clear validation error
Upload a PDF larger than the configured limit	Upload is rejected
Ask a question answered in a document	Relevant passages inform the answer
Ask about an absent specification	Assistant states that it could not verify the detail
Ask with an empty knowledge base	Assistant handles missing evidence without crashing
Stop Ollama	A clear local-service error is reported
Disable web search	No online research is claimed unless it actually occurs
Enable supported web search	Online research is attempted and available sources are attributed
Disconnect OpenRouter	A clear answer-generation error is reported
Ask a safety-critical question	Response emphasizes verification against official documentation

A representative test dataset should include questions with direct answers, paraphrased answers, missing information, conflicting sources, exact model numbers and safety-critical specifications.

17 Success Criteria

- Reliable PDF ingestion and persistent indexing.
- Consistent local embedding generation.
- Relevant retrieval of technical passages.
- Answers grounded in available evidence.
- Transparent attribution of internal and online sources.
- Graceful handling of missing evidence and unavailable services.
- A usable and responsive browser interface.
- No exposure of API credentials.
- Repeatable evaluation using documented test cases.

Quantitative targets for retrieval accuracy, answer quality, response latency and cost should be established after a baseline evaluation rather than assumed in advance.

18 Conclusion

The Industrial Product Intelligence Assistant demonstrates how local semantic search and hosted language models can be combined to improve access to industrial technical information. By integrating FastAPI, PyMuPDF, Ollama, ChromaDB and OpenRouter, it provides a foundation for conversational document search and AI-assisted product research.

Its principal value is turning lengthy technical documentation into a more accessible question-and-answer experience while preserving a distinction between source evidence and generated explanations.

The next development priority is to strengthen retrieval reliability, implement a verified web-search fallback, improve source citations and validate the application against realistic industrial product questions.

PROJECT	APPROACH	INFRASTRUCTURE
Industrial Product Intelligence Assistant	Retrieval-Augmented Generation	Ollama + ChromaDB (local) OpenRouter (answers)

Core objective: make industrial product documentation easier to search, understand and verify.